Source Code Analysis of the multi_brains Framework

Source Code Analysis of the multi_brains Framework

- 1. Course Content
- 2.Package Structure of the Source Code
  - 2.1 Package File Structure
- 3.Speech Recognition Function
  - 3.1 Dynamic Recording with Voice Activity Detection
  - 3.3 ASR Speech Recognition
- 4.Model Service Function
- 5. Action Server Function
- 6.Interruption Function
  - 6.1 Interruption During the Recording Stage
  - 6.2 Interruption During the Dialogue Stage
  - 6.3 Interruption During the Action Stage
    - 6.3.1 Ordinary Action Interruption
    - 6.3.2 Action Interruption with Subprocesses

1. Course Content

- Analyze the core functions of the multi_brains package architecture.
- In subsequent lessons, the action functions newly appearing in each tutorial will be explained individually.

> **Tip**
>
> - This lesson is a pure program-analysis lesson, for users who want a deeper understanding of the functions. If you only need to experience the functions, you can skip this lesson.

2.Package Structure of the Source Code

2.1 Package File Structure

```
├── config
│   ├── map_mapping.yaml
│   ├── multi_brains_setting.yaml
│   └── README.MD
├── language
│   ├── en.yaml
│   └── zh.yaml
├── launch
│   └── llm_agent_control.launch.py
├── multi_brains
│   ├── action_service.py
│   ├── asr_detect.py
│   ├── __init__.py
│   ├── model_service.py
│   └── utils
```

```
├── package.xml
├── resource
|   └── multi_brains
├── setup.cfg
├── setup.py
├── system_vioce
|   ├── en
|   ├── notify.mp3
|   ├── test_en.wav
|   ├── test_zh.wav
|   └── zh
```

- config

  The configuration folder, storing configuration **file templates**.

- multi_brains source code folder
- asr_detect.py

  The speech recognition program file.

- model_service.py

  The model server program file, used to call various model interfaces to implement the model inference architecture.

- action_service.py

  The action server program file, used to receive the action list requested by the model server and control the robot to move.

- The launch startup file folder, used to store the ros2 node launch files.
- language pack

  Stores log files of different languages.

- system_vioce

  System audio files.

3.Speech Recognition Function

Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/asr_detect.py
```

3.1 Dynamic Recording with Voice Activity Detection

`_run_streaming` is the core loop of **offline (local) streaming speech recognition**. Combined with VAD voice activity detection, it implements dynamic recording that "recognizes as you speak and stops automatically when you finish". Its processing flow is as follows:

- **Interruptible loop**: The main loop condition is `while not self._stop_rec_event.is_set()`. Once an external event (wake-up again) sets the stop event, the loop exits immediately. This is the basis for interrupting during the recording stage.

- **Fetch audio and resample**: Block and take one audio chunk from the audio queue `audio_q` (0.3s timeout), then use `resample_audio` to resample from the device sample rate to 16kHz for the recognition engine.
- **Pre-buffer to prevent losing the first word**: Before VAD triggers, store the most recent audio chunks in `pre_buffer` (at most 3 chunks, about 600ms); once VAD detects a human voice (`vad_triggered`), first feed the pre-buffer to the recognition engine in one go and then feed the current chunk, to avoid losing the beginning of the speech.
- **VAD status detection**: Call `_vad_engine.accept_waveform` + `is_detected()` to determine whether someone is speaking, and refresh the terminal prompt icon accordingly (`○` silent / `●` speaking).
- **Endpoint determines the end**: When `result.is_endpoint` is true and the recognized text is not empty, the sentence is considered finished, the final result `[✓]` is printed, and the loop exits; if it is an empty endpoint, `reset()` and continue waiting.
- **Dual timeout**: If no human voice is ever detected and `no_speech_timeout` is exceeded, it times out and exits; if a human voice has been detected but silence continues for more than 5 seconds, recording also ends.

```
    def _run_streaming(self, audio_q: queue.Queue, rec_start: float) -> str:
        """Local model streaming ASR processing"""
        if not self._streaming_engine:
            return ''
        speech_detected = False
        last_speech_time = time.time()
        final_text = ''
        prev_display = ''
        pre_buffer = []         # Pre-buffer: save audio before VAD detection to
avoid losing the beginning
        PRE_BUFFER_SIZE = 3     # Keep the first 3 audio chunks (approx. 600ms)
        vad_triggered = False  # Whether VAD has been triggered

        while not self._stop_rec_event.is_set():
            try:
                chunk_hw = audio_q.get(timeout=0.3)
            except queue.Empty:
                if not speech_detected:
                    if time.time() - rec_start > self.no_speech_timeout:
                        print()
                        self.get_logger().warn("No speech detected, timeout
exit.")
                        break
                continue

            # Resample to 16kHz
            chunk_16k = resample_audio(chunk_hw, self.device_sample_rate,
self.sample_rate)

            # -- VAD status detection --
            vad_icon = Fore.WHITE + '○' + Fore.RESET
            if self._vad_engine:
                try:
                    self._vad_engine.accept_waveform(chunk_16k)
                    if self._vad_engine.is_detected():
```

```
                        if not vad_triggered:
                            vad_triggered = True
                        speech_detected = True
                        last_speech_time = time.time()
                        vad_icon = Fore.GREEN + '●' + Fore.RESET
                    self._vad_engine.clear_segments()
                except Exception:
                    pass

            # -- Pre-buffer + ASR recognition --
            if vad_triggered:
                # VAD triggered: feed pre-buffer audio first, then current chunk
                if pre_buffer:
                    for buf_chunk in pre_buffer:
                        self._streaming_engine.accept_waveform(self.sample_rate,
buf_chunk)
                        self._streaming_engine.decode()
                    pre_buffer.clear()
                self._streaming_engine.accept_waveform(self.sample_rate,
chunk_16k)
                self._streaming_engine.decode()
            else:
                # VAD not triggered: save to pre-buffer
                pre_buffer.append(chunk_16k)
                if len(pre_buffer) > PRE_BUFFER_SIZE:
                    pre_buffer.pop(0)

            result = self._streaming_engine.get_result()
            result_text = result.text.lower() if self.language == "en" else
result.text

            display = self._format_display(f" {vad_icon} ", result_text)
            if display != prev_display:
                # Use ANSI to clear remaining characters at end of line
                print(f"\r{display}\033[K", end='', flush=True)
                prev_display = display

            # Endpoint detected, recording finished
            if result.is_endpoint:
                if result_text.strip():
                    final_text = result_text
                    print(f"\r[\u2713] {final_text}\033[K", flush=True)
                    break
                else:
                    # Empty endpoint: reset and continue waiting
                    self._streaming_engine.reset()
                    prev_display = ''
                    continue

            # Post-speech silence timeout
            if speech_detected and (time.time() - last_speech_time > 5.0):
                result = self._streaming_engine.get_result()
                result_text = result.text.lower() if self.language == "en" else
result.text
                final_text = result_text
                print()
```

```
                break

        # Ensure newline
        if final_text == '' and prev_display:
            print()

        return final_text
```

3.3 ASR Speech Recognition

`_recognition_session` is the entry point of **one complete recording-and-recognition session**. It is executed by the recording thread created after wake-up and is responsible for the complete chain of "capture audio → call recognition → publish result":

- **Audio capture (producer)**: Use `sounddevice.InputStream` to open the microphone; the callback `_audio_cb` converts the int16 audio to float32 and pushes it into the queue `audio_q` for the recognition loop to consume.
- **Recognition method selection**: According to `use_oline_asr`, call either online recognition `_run_online_asr` or local streaming recognition `_run_streaming`; both take audio from the same `audio_q`.
- **Resource release**: In `finally`, regardless of success or failure, `stream_alive[0]=False` is set and the audio stream is `stop()` / `close()`d, ensuring the microphone is correctly released.
- **Result filtering and publishing**: After stripping the recognized text, if its length is less than the threshold `ASR_THREASHOLD`, it is considered invalid (too short) and discarded directly; otherwise it is published to the `asr_result` topic through `asr_pub`, and forwarded to model_service through `llm_request_pub` with `LlmRequest(robot_feedback=False, llm_request=text)` for Dify to handle. Here `robot_feedback=False` indicates that this is a new user instruction, rather than feedback from the robot's action execution.

```
    def _recognition_session(self):
        chunk_size = int(self.device_sample_rate * self.CHUNK_DURATION)
        audio_q: queue.Queue = queue.Queue()
        stream_alive = [True]

        def _audio_cb(indata, frames, time_info, status):
            """sounddevice callback: put int16 chunk into queue"""
            if not stream_alive[0]:
                return
            chunk = indata[:, 0].astype(np.float32) / 32768.0
            audio_q.put(chunk)

        try:
            stream = sd.InputStream(
                samplerate=self.device_sample_rate,
                channels=1,
                dtype=np.int16,
                blocksize=chunk_size,
                callback=_audio_cb)
```

```
            stream.start()
        except Exception as exc:
            self.get_logger().error(Fore.RED + f"Audio stream open failed:
{exc}"+ Fore.RESET)
            return

        rec_start = time.time()
        try:
            if self.use_oline_asr:
                # Online ASR
                result_text = self._run_online_asr(audio_q, rec_start)
            else:
                # Local streaming ASR
                result_text = self._run_streaming(audio_q, rec_start)
        finally:
            stream_alive[0] = False
            try:
                stream.stop()
                stream.close()
            except Exception:
                pass

        # Recognition result
        result_text = result_text.strip() if result_text else ''
        if len(result_text)< self.ASR_THREASHOLD:
            self.get_logger().info(f"ASR result: {result_text} < ASR_THRESHOLD!")
            return
        self.asr_pub.publish(String(data=result_text))
        # Forward the valid recognition result to model_service for Dify
processing
        self.llm_request_pub.publish(LlmRequest(robot_feedback=False,
llm_request=result_text))
```

4.Model Service Function

Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/model_service.py
```

Explanation of the core functions:

- Check in an independent thread whether there is a request to access the model in the `llm_handler_queue` queue.
- Once an element is pushed into the queue, call Dify's access interface `chat` function with different parameters according to the request type.
- After obtaining the AI agent's reply from the Dify side, parse the corresponding content to play the voice and send the action list to the action server for execution.

```
    def llm_request_callback(self, msg:LlmRequest):
        '''Topic callback function, receive model request and put into queue'''
```

```
        if self.debug_mode: self.get_logger().info(f"robot_feedback:
{msg.robot_feedback},llm_request:{msg.llm_request}")
        if msg.robot_feedback:
            if msg.llm_request =="Robot image feedback complete":
                self._enqueue_request([msg.llm_request,'image_request',True])
            elif msg.llm_request =="finish":
                self.clear_request_queue()
                self.dify_llmclient.reset_conversation()
            else:
                if self.debug_mode: self.get_logger().info("Normal robot
feedback")
                self._enqueue_request([msg.llm_request,'text_request',True])

        else:# model requests from other sources
            self._enqueue_request([msg.llm_request,'text_request',None])

    def _enqueue_request(self, request:list):
        '''Non-blocking enqueue, drop the request if the queue is busy to avoid
blocking the spin thread'''
        try:
            self.llm_handler_queue.put_nowait(request)
        except queue.Full:
            self.get_logger().warn(Fore.YELLOW+f"llm_handler_queue is busy,
request dropped: {request[0]}"+Fore.RESET)

    def handle_llm_thread(self)->None:
        '''Handle model request'''
        while True:
            try:
                if self.llm_handler_queue.empty():
                    time.sleep(1.0)#sleep 1 second when there is no request
                    continue

                request_query = self.llm_handler_queue.get()
                if self.debug_mode: self.get_logger().info(f"Processing LLM
request: {request_query}")

                if request_query[1]=='text_request':
                    '''text request'''

 result=self.dify_llmclient.chat(request_query[0],robot_feedback=request_query[2]
)
                elif request_query[1]=='image_request':
                    '''vision + text request'''

 result=self.dify_llmclient.chat(request_query[0],image_path=self.image_cache_pat
h,robot_feedback=request_query[2])
                else:
                    self.get_logger().error(Fore.RED+f"Unknown request type:
{request_query[1]}"+Fore.RESET)
                    continue

                if result[0]:
                    split_result=self.extract_actions(result[1])
                    if split_result is None: continue
                    action_list,llm_response,decision_plan=split_result
```

```
                    if decision_plan is not None:
                        self.get_logger().info(Fore.YELLOW+f"Decision-making
level planning: {decision_plan}"+Fore.RESET)
                    self.get_logger().info(Fore.YELLOW+f'"action":
{action_list},"response": {llm_response}'+Fore.RESET)

                    if self.use_tts:
                        if
self.tts_engine.synthesize(llm_response,self.tts_out_path) :
                            self.play_audio(self.tts_out_path)
                        else:
                            self.get_logger().error(Fore.RED+"Speech synthesis
failed. Check whether the TTS model is available"+Fore.RESET)
                    if decision_plan is not None:
                        self.text_pub.publish(String(data=f"Decision-making level
planning: {decision_plan}"))
                    self.text_pub.publish(String(data=f'"action": {action_list},
"response": {llm_response}'))

                    if action_list!=[]: self.send_action_service(action_list,
llm_response)
                else:
                    error_msg = f"The dify request failed. Error Log:{result[1]}"
                    self.get_logger().error(Fore.RED+error_msg+Fore.RESET)
                    self.text_pub.publish(String(data=f"[ERROR] {error_msg}"))
            except Exception as e:
                self.get_logger().error(Fore.RED+f"handle_llm_thread exception:
{e}\n{traceback.format_exc()}"+Fore.RESET)
                time.sleep(0.5)
```

5. Action Server Function

- Program source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service.p
y
```

- It contains the implementation of all the basic action functions that the robot can execute: receiving an action list request and executing the corresponding actions. It has the function of being interrupted upon being woken up again.

Parsing of the core callback function `execute_callback`:

`execute_callback` is the execution callback of the `/action_service` (Rot action) server, responsible for receiving the **action string list** issued by model_service and parsing, reflectively invoking, and summarizing the feedback one by one:

- **State marking**: On entry, set `action_runing=True`, `action_state="RUNNING"`, and clear this round's result records `action_results`.

- **Execute one by one**: Iterate over `actions`; before executing each action, first check whether `interrupt_event` is set (interrupted by wake-up); if so, `break` out and no longer execute the subsequent actions.
- **Parsing and reflective invocation**: Use `_parse_action` to parse the action string into "function name + argument list". If parsing fails, record it as FAILED and skip; then reflectively call the corresponding action function through `getattr(self, action_name)(*args)` (i.e., an action registered by `@llm_action`).
- **Unified return protocol**: Compatible with three kinds of return values — `None` (no-op/self-feedback actions, such as seewhat, stop_feedback, finish_task), `[bool, str]` (success or not + result message), `True` (success). Based on this, accumulate `success_actions` / `failed_actions` and the details.
- **Result feedback**: After the loop ends, if interrupted, set the state to `INTERRUPTED`; otherwise summarize the success/failure as a `Robot feedback:` text, publish it to the `llm_request_handler` topic through `_publish_feedback`, and feed it back to model_service, which finally feeds it back to the Dify agent. Finally, reset `action_runing`, clear `interrupt_event`, and call `goal_handle.succeed()`.
- `stop()`: Publishes an all-zero `Twist` to `cmd_vel`, making the chassis stop immediately.
- `_execute_action(twist, durationtime)`: Continuously publishes `Twist` at about 10Hz within the `durationtime` duration to achieve timed motion; as soon as the loop detects that `interrupt_event` is set, it immediately `break`s and stops publishing, realizing interruptible chassis motion.

```
    def execute_callback(self, goal_handle):
        actions = goal_handle.request.actions
        if self.debug_mode: self.get_logger().info(f"Entered action execution
callback, preparing to execute action list: {actions}")
        self.action_runing = True
        self.action_state = "RUNNING"
        self.action_results = []
        success_actions = []
        failed_actions = []
        failed_detail = ""
        success_details = []
        for action in actions:
            if self.interrupt_event.is_set():
                break
            failed_detail = ""
            action_name, args, parse_error = self._parse_action(action)
            if action_name is None:
                self.get_logger().error(Fore.RED+f"action_service: {parse_error},
skip execution"+Fore.RESET)
                failed_actions.append(action)
                failed_detail = parse_error
                self._record_result(action, "FAILED", parse_error)
                continue
            try:
                method = getattr(self, action_name)
                action_result = method(*args)
            except Exception as e:
```

```
                self.get_logger().error(Fore.RED+f"action_service: {action}
raised exception: {e}"+Fore.RESET)
                failed_actions.append(action)
                failed_detail = str(e)
                self._record_result(action, "FAILED", str(e))
                continue
            if action_result is None:
                #None means no-op or self-feedback action
                self._record_result(action, "SELF_FEEDBACK")
            elif isinstance(action_result, (list, tuple)) and len(action_result)
== 2:
                #[bool, str] protocol with result message
                detail = str(action_result[1])
                if action_result[0]:
                    success_actions.append(action)
                    if detail:
                        success_details.append(detail)
                    self._record_result(action, "SUCCEEDED", detail)
                else:
                    failed_actions.append(action)
                    failed_detail = detail
                    self._record_result(action, "FAILED", detail)
            elif action_result is True:
                success_actions.append(action)
                self._record_result(action, "SUCCEEDED")
            else:
                failed_actions.append(action)
                self._record_result(action, "FAILED")

        if self.interrupt_event.is_set():
            self.action_state = "INTERRUPTED"
        else:#to dify-agent
            feedback = ""
            if failed_actions:
                feedback = f"Robot feedback: Failed to execute {failed_actions}"
                if failed_detail:
                    feedback += f", error: {failed_detail}"
            elif success_actions:
                feedback = f"Robot feedback: Execution {success_actions}
completed"
                if success_details:
                    feedback += f", result: {'; '.join(success_details)}"

            if feedback:
                self._publish_feedback(feedback)
            else:
                if self.debug_mode: self.get_logger().info("No operation, no
feedback.")

        if self.debug_mode: self.get_logger().info("The exit action executes the
callback function.")
        self.action_runing = False
        if self.action_state == "RUNNING":
            self.action_state = "FAILED" if failed_actions else "SUCCEEDED"
        self.interrupt_event.clear()
        goal_handle.succeed()
```

```
        result = Rot.Result()
        result.success = True
        return result


    def stop(self):
        twist = Twist()
        twist.linear.x = 0.0
        twist.linear.y = 0.0
        twist.angular.z = 0.0
        self.cmd_vel_pub.publish(twist)


    def _execute_action(self, twist, durationtime:float=3.0):
        '''Internal function to execute action '''
        start_time = time.time()
        while (time.time() - start_time) < durationtime:
            if self.interrupt_event.is_set():
                break
            self.cmd_vel_pub.publish(twist)
            time.sleep(0.1)
```

6.Interruption Function

The robot supports being interrupted by being woken up again at **any stage of recording, dialogue, or action**. The entire interruption mechanism is accomplished through the collaboration of three nodes:

- **asr_detect.py (speech recognition node VoiceProcessor)**: Detects hardware/keyboard wake-up, publishes the `wakeup_signal` topic, manages the interruption and restart of the recording thread, and plays the wake-up response audio.
- **model_service.py (model service node Ros_LLM_Service)**: Calls Dify and TTS to synthesize speech, and plays the reply speech and issues the action list through the `play_audio` action service.
- **action_service.py (action service node ActionController)**: Subscribes to `wakeup_signal`, and interrupts the action being executed through `interrupt_event` and performs an emergency stop.

> The interruption signal uniformly comes from "wake-up": every time asr_detect wakes up, it publishes `wakeup_signal` (Bool=True) for action_service to trigger the action interruption; the interruption of the recording thread itself is accomplished by `_stop_rec_event` inside asr_detect.

6.1 Interruption During the Recording Stage

If you find that you misspoke during recording, or you are not satisfied with what you said and need to re-record, you can directly wake up again **during the recording process** to interrupt the previous recording and start recording again.

- The logic is implemented in the `run()` main loop of the `VoiceProcessor` class in asr_detect.py:

- Each time a wake-up is detected (hardware `_hw_wakeup_event` or keyboard `_kb_wakeup_event`):
    1. Clear the wake-up event and publish `wakeup_signal` (Bool=True) to notify other nodes;
    2. If the wake-up response audio is enabled, call `wakeup_voice()` to respond to the user;
    3. If there is already a running recording thread `_current_rec_thread`, set `_stop_rec_event` to notify it to exit, `join(timeout=3.0)` to wait for it to end, and then clear the event;
    4. Reset the state of the streaming recognition engine and the VAD engine;
    5. Start a new `_recognition_session` recording thread and begin a new round of recording.
- The main loop conditions of the recording recognition loops (`_run_streaming` / `_run_online_asr`) are both `while not self._stop_rec_event.is_set()`, so once `_stop_rec_event` is set, the current recognition ends immediately, realizing interruption during the recording stage.

```
    def run(self):
        while rclpy.ok():
            hw_woke = self._hw_wakeup_event.wait(timeout=0.05) # voice wakeup
            kb_woke = self._kb_wakeup_event.is_set() # Keyboard wakeup
            if hw_woke or kb_woke:
                self._hw_wakeup_event.clear()
                self._kb_wakeup_event.clear()
                self.wakeup_pub.publish(Bool(data=True))
                if self.enable_wakeup_voice:
                    self.wakeup_voice()  # play wakeup audio before recording

                if (self._current_rec_thread is not None and
self._current_rec_thread.is_alive()):
                    self._stop_rec_event.set()
                    self._current_rec_thread.join(timeout=3.0)
                    self._stop_rec_event.clear()

                # Reset engine state
                if self._streaming_engine:
                    self._streaming_engine.reset()
                if self._vad_engine:
                    self._vad_engine.reset()
                    self._vad_engine.clear_segments()

                # Start new recognition session thread
                self._current_rec_thread = threading.Thread(
                    target=self._recognition_session, daemon=True)
                self._current_rec_thread.start()
            time.sleep(0.02)
```

6.2 Interruption During the Dialogue Stage

If, while the robot is speaking (playing the reply speech), you are not satisfied with the reply or do not want it to continue, you can use the wake word to directly interrupt the robot's speech and start recording; at this point you can give the robot a new instruction (still within the current task cycle), or say "end the current task" to end the current task and start a new task cycle.

- The robot's reply speech is played by `play_audio()` in model_service.py: as an **action client**, it sends the audio file path to the common action server through the `play_audio` (PlayAudio) action (the `common_service` node, which plays it internally using pygame), and blocks with `done_event.wait()` waiting for playback to finish.
- The `run()` of asr_detect.py publishes `wakeup_signal` on every wake-up and calls `wakeup_voice()` — it also acts as a client, sending a new playback goal (the wake-up response audio) to the `play_audio` action service. When the play_audio server detects cancellation/preemption in the playback thread (`goal_handle.is_cancel_requested` or `cancel_play_audio`), it calls `pygame.mixer.music.stop()` to stop the current playback, thereby interrupting the reply speech being played.
- After interruption, a new recording session starts immediately, and the user can continue to issue new instructions.
- If the user wants to end the current task cycle: when the action layer executes `finish_task()`, it publishes feedback with the content `finish`; after model_service's `llm_request_callback` receives it, it clears the request queue and calls `reset_conversation()` to reset the Dify session context.

model_service.py plays the reply speech (sends the play_audio action goal and blocks waiting for playback to finish):

```
    def play_audio(self,file_path: str) -> None:
        '''Play audio through the play_audio action service and wait until
playback finishes'''
        if not self.audio_client.wait_for_server(timeout_sec=3.0):
            self.get_logger().warn(Fore.YELLOW+"play_audio action server not
available, skip audio playback"+Fore.RESET)
            return
        goal = PlayAudio.Goal()
        goal.file_path = file_path
        done_event = threading.Event()

        def _result_callback(future):
            result = future.result().result
            if not result.success:
                self.get_logger().warn(f"play_audio finished abnormally:
{result.message}")
            done_event.set()

        def _goal_response_callback(future):
            goal_handle = future.result()
            if not goal_handle.accepted:
                self.get_logger().warn("play_audio goal rejected by action
server")
                done_event.set()
                return
```

```
            goal_handle.get_result_async().add_done_callback(_result_callback)

        send_future = self.audio_client.send_goal_async(goal)
        send_future.add_done_callback(_goal_response_callback)
        done_event.wait()  #Block until audio playback finishes
```

asr_detect.py plays the wake-up response audio (sends a new goal to the same play_audio action service, thereby interrupting the previous speech):

```
    def wakeup_voice(self):
        """Play a random wakeup audio and block until playback finishes."""
        if not self.wakeup_audio_list:
            return
        audio_file = random.choice(self.wakeup_audio_list)

        if not self.audio_client.wait_for_server(timeout_sec=1.0):
            self.get_logger().warn("play_audio action server not available, skip
wakeup voice")
            return

        goal = PlayAudio.Goal()
        goal.file_path = audio_file

        done_event = threading.Event()
        def _result_cb(future):
            done_event.set()

        def _goal_response_cb(future):
            goal_handle = future.result()
            if not goal_handle.accepted:
                self.get_logger().warn("play_audio goal rejected")
                done_event.set()
                return
            result_future = goal_handle.get_result_async()
            result_future.add_done_callback(_result_cb)

        future = self.audio_client.send_goal_async(goal)
        future.add_done_callback(_goal_response_cb)

        done_event.wait(timeout=10.0)
```

6.3 Interruption During the Action Stage

If the robot is woken up while executing an action, it will stop the current action and restore the initial pose. Specifically, it can be divided into ordinary action interruption and action interruption with subprocesses.

The unified entry point for all action interruptions is `wakeup_callback` in action_service.py: it subscribes to the `wakeup_signal` topic, and when it receives a wake-up signal while an action is running (`action_runing` is true), it sets `interrupt_event` and calls `_emergency_stop()` to perform an immediate emergency stop.

```
    def wakeup_callback(self, msg:Bool):
        ''' Voice wake-up callback function '''
        self.get_logger().info("wakeup received")
        if msg.data:
            if self.action_runing:
                self.interrupt_event.set()
                self._emergency_stop()

    def _emergency_stop(self):
        '''Immediately stop all physical motion when wakeup is received'''
        # 1. Stop chassis
        self.stop()
        # 2. Cancel navigation goal
        if self.nav_goal_handle is not None:
            try:
                self.nav_goal_handle.cancel_goal_async()
            except Exception:
                pass
        # 3. Cancel target tracking
        if self.cancel_track_client.service_is_ready():
            try:
                self.cancel_track_client.call_async(CancelTrack.Request())
            except Exception:
                pass
        # 4. Stop visual track service
        if self.visual_track_client.service_is_ready():
            try:
                self.visual_track_client.call_async(SetBool.Request(data=False))
            except Exception:
                pass
        # 5. Kill subprocesses
        self._stop_aruco_detect()
        self._stop_color_detect()
        self._stop_follow_line_clear()
        self.get_logger().info("Emergency stop executed")
```

`_emergency_stop()` stops the chassis, cancels the navigation goal, cancels target tracking, shuts down the visual tracking service, and ends all detection subprocesses at once; it is the fastest emergency stop method. The `interrupt_event` flag, on the other hand, is checked by each action function in its waiting loop to achieve graceful exit and state recovery.

6.3.1 Ordinary Action Interruption

Ordinary actions refer to chassis motion, robotic arm motion, navigation, visual alignment, and other motions controlled via topics/actions/services. They achieve interruption by checking `interrupt_event` in their waiting loops:

- **Timed chassis motion**: `_execute_action` checks `interrupt_event` in the loop that continuously publishes `cmd_vel`; if it is set, it immediately `break`s and stops publishing.
- **Chassis odometry motion / navigation / visual alignment**: When `_move_by_odom`, `navigation`, `_precise_adjust`, `_adjust_grasp_pose`, etc. detect an interruption in the loop waiting for the action result, they call `goal_handle.cancel_goal_async()` to cancel the goal, `stop()` to stop the car, and return failure.

- **Robotic arm action group**: `_group_shake` / `_group_nod` / `_group_applaud` / `_group_dance` check `interrupt_event` between each step; when interrupted, they publish the initial pose `init_joints` to retract the robotic arm and then return.
- **Action scheduling layer**: `execute_callback` also checks `interrupt_event` before executing each action; if it is set, it no longer executes the subsequent actions and marks the state as `INTERRUPTED`.

The interruptible implementation of timed chassis motion (stops publishing velocity immediately when an interruption is detected):

```
    def _execute_action(self, twist, durationtime:float=3.0):
        '''Internal function to execute action '''
        start_time = time.time()
        while (time.time() - start_time) < durationtime:
            if self.interrupt_event.is_set():
                break
            self.cmd_vel_pub.publish(twist)
            time.sleep(0.1)
```

Taking the odometry motion `_move_by_odom` as an example, when an interruption is detected while waiting for the action result, it cancels the goal and stops the car:

```
        result_future = goal_handle.get_result_async()
        while rclpy.ok() and not result_future.done():
            if self.interrupt_event.is_set():
                try:
                    goal_handle.cancel_goal_async()
                except Exception:
                    pass
                self.stop()
                return [False, "MoveByOdom interrupted by wakeup"]
            time.sleep(0.2)
```

The robotic arm action group detects interruption between each step and retracts the robotic arm (taking `_group_shake` as an example):

```
    def _group_shake(self):
        for _ in range(3):
            if self.interrupt_event.is_set():
                self.pubSix_Arm(self.init_joints)
                return None
            if not self.pubSix_Arm([140, 130, 0, 5, 90, 0]):
                return [False, "Failed to publish arm joints during shake"]
            time.sleep(1.0)
            if not self.pubSix_Arm([40, 130, 0, 5, 90, 0]):
                return [False, "Failed to publish arm joints during shake"]
            time.sleep(1.0)
        if not self.pubSix_Arm(self.init_joints):
            return [False, "Failed to publish arm joints during shake"]
        return [True, ""]
```

> **Note**

> Unlike the older version: the robotic arm publishing function `pubSix_Arm` itself **no longer checks** the interruption flag; the interruption determination and initial pose recovery are handled uniformly in the respective action group loops.

6.3.2 Action Interruption with Subprocesses

Actions such as AprilTag marker grasping/removal (`GraspArUco` / `ArUcoRemoveHigh`), color block grasping (`GraspColor`), and line-following obstacle clearing (`follow_line_clear`) need to start external detection/control programs (such as `aruco_detect`, `color_detect`, and `follow_line_clear` of common_demo) in an **independent subprocess**. Here, `GraspArUco` is used as an example for explanation:

- `_start_aruco_detect()` uses `subprocess.Popen` to start the detection subprocess, and then `_wait_for_aruco_target()` polls for the target in a waiting loop; this loop checks `interrupt_event.is_set()`, and if interrupted, returns `None`, and the action then fails and exits.
- When an interruption occurs, `_emergency_stop()` calls `_stop_aruco_detect()` / `_stop_color_detect()` / `_stop_follow_line_clear()`, which use `kill_process_tree(pid)` to **recursively end the entire subprocess tree**, avoiding leftover processes.
- In addition, the action function uses `try/finally` to ensure that, whether it ends normally or is interrupted, it calls the corresponding `_stop_*` in `finally` to clean up the subprocess.

```
    @llm_action
    def GraspArUco(self, target_id) -> list[bool, str]:
        try:
            target_id = int(str(target_id).strip().strip("'\""))
        except (TypeError, ValueError):
            return [False, f"GraspArUco failed: invalid target_id '{target_id}',
must be an integer"]

        self.pubSix_Arm(self.aruco_detect_pose)
        time.sleep(2.0)
        try:
            #step 1: launch the standalone detect process targeting the requested
ID
            self._start_aruco_detect(target_id=target_id)
            if self._wait_for_aruco_target(target_id, timeout_sec=15.0) is None:
                return [False, f"AprilTag with ID {target_id} not detected in
current scene"]
            # ...step 2~4: align to target / compute grasp point / execute grasp
(unrelated to interruption, omitted here)...
            return [True, f"AprilTag {target_id} grasped successfully"]
        finally:
            self._stop_aruco_detect()
```

Checking the interruption flag in the loop waiting for the detection target (`_wait_for_aruco_target`), and recursively ending the subprocess tree (`_stop_aruco_detect`):

```
    def _stop_aruco_detect(self):
        """Terminate the standalone aruco_detect process if still running."""
        if self._aruco_process is not None:
```

```
            if self._aruco_process.poll() is None:
                kill_process_tree(self._aruco_process.pid)
            self._aruco_process = None

    def _wait_for_aruco_target(self, target_id, timeout_sec=15.0):
        """Wait until the detect process publishes /aruco_target for
target_id."""
        deadline = time.time() + timeout_sec
        while rclpy.ok() and time.time() < deadline:
            if self.interrupt_event.is_set():
                return None
            with self._aruco_target_lock:
                msg = self._aruco_target_cache.get(int(target_id))
            if msg is not None:
                return msg
            time.sleep(0.2)
        return None
```